Module 38

Algorithms and Data Structures/3: Data Structures

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 17, 2026

Module Recap

- Introduced Data Structures
- Defined Linear Data Structure
- Reviewed array, list, stack, queue
- Reviewed linear and binary search

Module Objectives

- Introducing Non-linear Data Structures - graph, tree, hash table
- Exploring Binary Search Tree
- Comparing Linear and Non-Linear Data Structures

Module Outline

- Non-linear Data Structures
- Binary Search Trees
- Comparison of Linear and Non-Linear Data Structures

Data Structure

Data Structure

- **Data structure** : A data structure specifies the way of organizing and storing in-memory data that enables efficient access and modification of the data.
- Linear Data Structures vs Non-linear Data Structures
- Most data structure has a container for the data and typical operations that its needs to perform
- For applications relating to data management, the key operations are:
  - Create
  - Insert
  - Delete
  - Find / Search
  - Close
- Efficiency is measured in terms of time and space taken for these operations

Non-linear Data Structures

Non-linear Data Structures: Why?

Summary observations for linear structures:

- All of them have the space complexity $\mathcal{O}(n)$, which optimal. However, the actual used space may be lower in array while linked list has an overhead of 100% (double)
- All of them offer satisfactory complexity for some operations while being unsatisfactory on the others.

	Array *Unordered*	**Array** *Ordered*	**Linked List** *Unordered*	**Linked List** *Ordered*
Access	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Delete	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Search	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Non-Linear data structures . . .

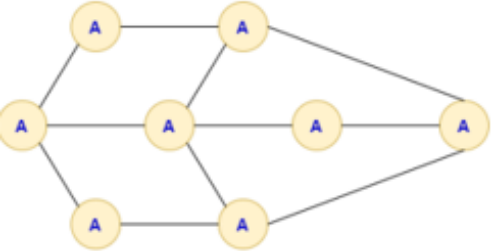
can be used to trade off between extremes and achieve a balanced good performance for all

Non-linear Data Structures (2)

- Nonlinear data structures are those data structures in which data items are not arranged in a sequence and each element may have multiple paths to connect to other elements.
- Traversing through the elements is not possible in one run as the data is non-linearly arranged.
- Common Non-Linear Data Structures include:
  - **Graph** : Undirected or Directed, Unweighted or Weighted, and variants
  - **Tree** : Rooted or Unrooted, Binary or n-ary, Balanced or Unbalanced, and variants
  - **Hash Table** : Array with lists (coalesced chains) and one or more hash functions
  - **Skip List** : Multi-layered interconnected linked lists

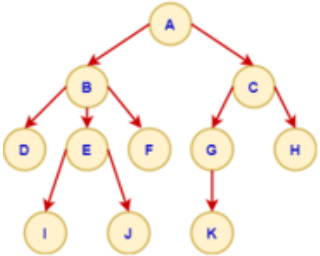
Non-linear Data Structures (3): Graph

- **Graphs** : Graph G is a collection of vertices V (store the elements) and connecting edges (links) E between vertices: $G = \langle V, E \rangle$ where $E \subseteq V \times V$
- A graph may be: Undirected or Directed, Unweighted or Weighted, Cyclic or Acyclic, Disconnected or Connected.
- Examples of a graph include: ER Diagram, Networks (Electrical, Water), Friendships in Facebook, Knowledge Graph.



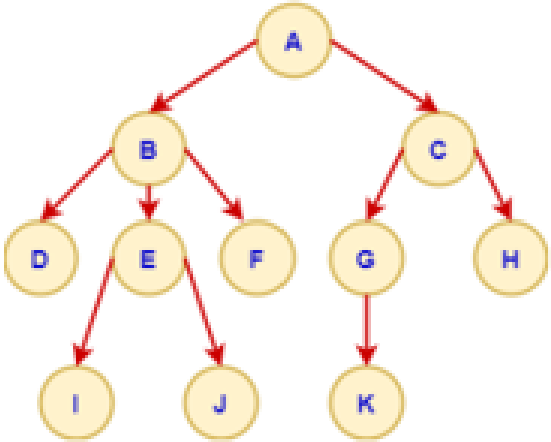
Non-linear Data Structures (4): Tree

- **Tree** : Is a connected acyclic graph representing hierarchical relationship
- A tree may be: Rooted or Unrooted, Binary or n-ary, Balanced or Unbalanced, Disconnected (forest) or Connected.
- Examples of a tree include: Composite Attributes, Family Genealogy, Search Trees, etc.



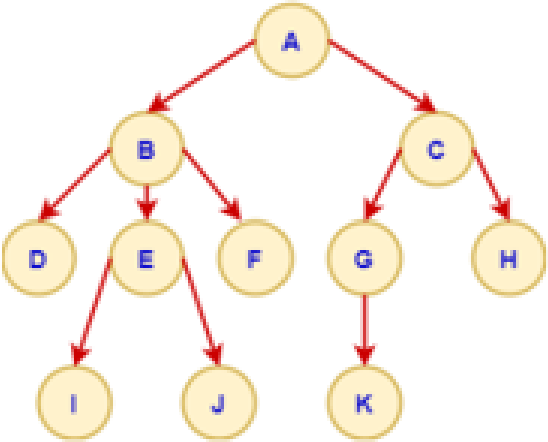
Non-linear Data Structures (5): Tree

- **Root** : The node at the top of the tree is called root.
- **Parent** : The node which is a predecessor of any node. Every node, except Root, has a unique parent.
- **Child** : A node which is the descendant of a node.
- **Leaf** : A node which does not have any child node.



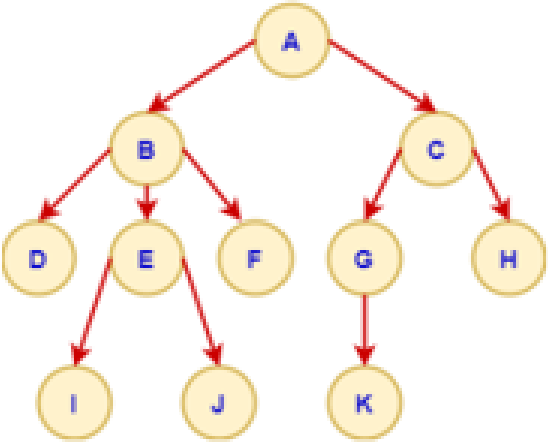
Non-linear Data Structures (6): Tree

- **Internal Nodes** : The node which has at least one child.
- **Subtree** : Subtree represents the tree rooted at that node.
- **Path** : Sequence of nodes along the edges of a tree.
- **Siblings** : Nodes having the same parent.
- **Arity** : Number of children of a node. Maximum arity of a node is defined as the arity of the tree.



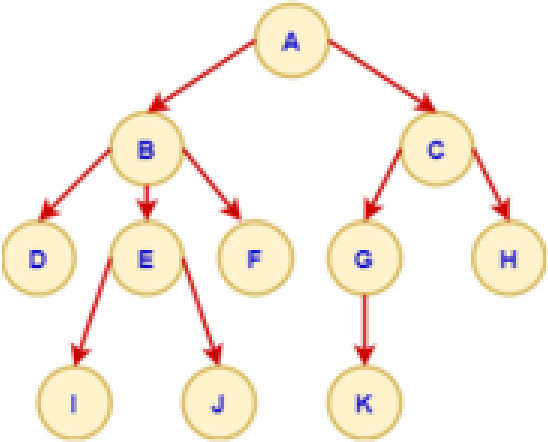
Non-linear Data Structures (7): Tree

- **Levels** : The root node is said to be at Level 0, children at Level 1, etc.
- **Level** is the length of the path or distance of a node from the root node.
- **Height** : Maximum level in a tree.
- **Binary Tree** : a tree where each node can have at most 2 children (arity 2).



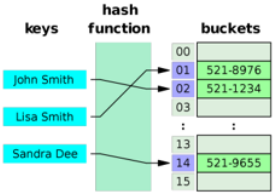
Non-linear Data Structures (8): Tree

- **Fact 1** : A tree with n nodes has $n-1$ edges
- **Fact 2** : The maximum number of nodes at level l of a binary tree is 2^l.
- **Fact 3** : If h is the height of a binary tree of n nodes, then:
  - $h+1 \leq n \leq 2^{h+1}-1$
  - $\lceil \lg(n+1) \rceil - 1 \leq h \leq n-1$
  - $\mathcal{O}(\log n) \leq h \leq \mathcal{O}(n)$
- For a k-ary tree, $\mathcal{O}(\log_k n) \leq h \leq \mathcal{O}(n)$



Non-linear Data Structures (9): Hash Table

- **Hash Table (Hash Map)** : implements an associative array abstract data type, a structure that can map keys to values by using a hash function to compute an index (hash code), into an array of buckets or slots, from which the desired value can be found.
- A hash table may be using Static or Dynamic Schemes, Open Addressing, 2-Choice Hashing, etc.
- Examples include Associative arrays, Database indexing, Caches, etc.



Binary Search Tree

Binary Search and Binary Search Tree

- During the study of linear data structure, we observed that:
  - Binary search is efficient in search of a key: $\mathcal{O}(\log n)$. However, it needs to be performed on a sorted array, and the array makes insertion and deletion expensive at $\mathcal{O}(n)$.
  - The linked list is efficient in insertion and deletion at $\mathcal{O}(1)$, while it makes the search expensive at $\mathcal{O}(n)$.
- Using non-linearity, specifically (binary) trees, we can combine the benefits of both.

Binary Search and Binary Search Tree (2)

- Note that once an array is sorted, we know the order in which its elements may be checked (for any key) during a search.
- As the binary search splits the array, we can conceptually consider the Middle Element to be the Root of a tree and the left/right sub-array to be its left/right sub-tree.
- Progressing recursively, we have a Binary Search Tree.

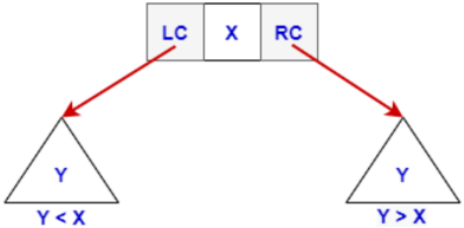
- Consider the data set:

10	12	14	17	19	22	25
LL	L	LR	M	RL	R	RR

- Search order is:
  - First: M
  - Second: L or R
  - Third:
    - For L: LL or LR
    - For R: RL or RR
  - Recur ...
- Put as a tree:

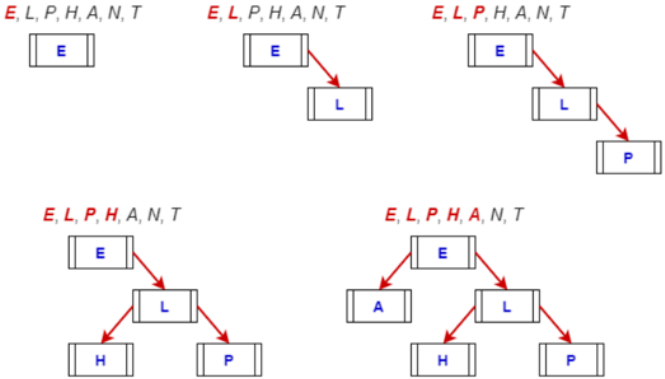
Binary Search Tree

- **Binary Search Tree (BST)** : Is a tree in which all the nodes hold the following:
  - The value of each node in the left sub-tree is less than the value of its root.
  - The value of each node in the right sub-tree is greater than the value of its root.
- **Structure of BST node** : Each node consists of an element, a link to the left child or the left subtree, and a link to the right child or the right subtree.



Binary Search Tree (2)

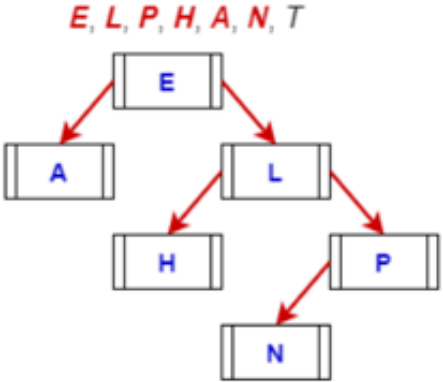
- **Example**: Obtain the BST by inserting the following values- *E, L, P, H, A, N, T*.

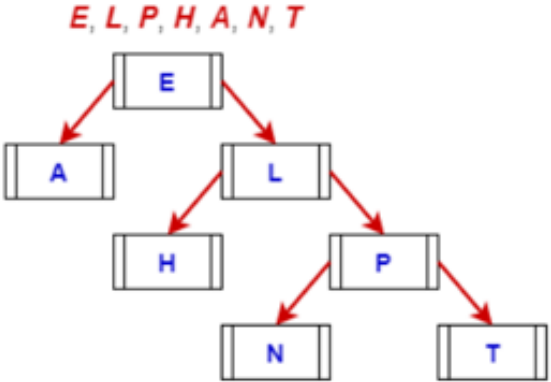


Binary Search Tree (3)

Example

Obtain the BST by inserting the following values E, L, P, H, A, N, T.

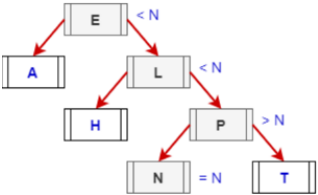




Searching a key in BST

search(root, key)

1. Compare the key with the element at root.
2. If the key is equal to root's element then Element found and return
3. else if the key is lesser than the root's element, `search(root.lc)` #search on the left subtree
4. else: `search(root.rc)` #search on the right subtree



Searching a key in BST (2)

Searching a key in a BST is $\mathcal{O}(h)$, where h is the height of the tree.

Worst Case

- The BST is a skewed binary search tree (can happen if keys are inserted in sorted order).
- Height (h) of the BST having n elements becomes $n - 1$.
- Time complexity of search in BST becomes $\mathcal{O}(n)$.

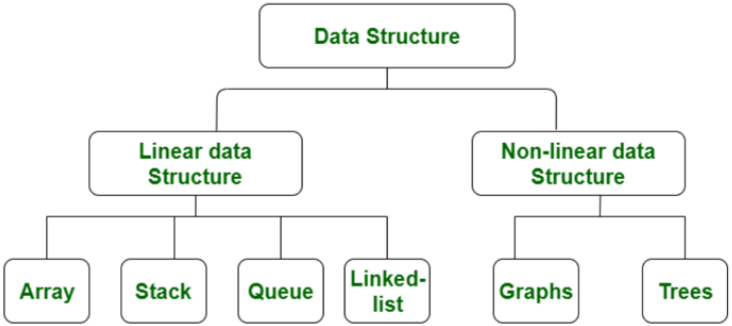
Best Case

- The BST is a balanced binary search tree (possible if keys are inserted in purely randomized order, or the tree is explicitly balanced).
- Height (h) of the binary search tree becomes $\log n$.
- Time complexity of search in BST becomes $\mathcal{O}(\log n)$.

Comparison of Linear and Non-Linear Data Structures

Linear and Non-Linear Data Structures

Module 38
Partha Pratim Das
Objectives & Outline
Data Structure
Non-linear Data Structures
Graph
Tree
Hash Table
Binary Search Tree
Build a BST
Search a Key
Comparison
Module Summary



Source: Difference between Linear and Non-linear Data Structures (11-Aug-2021)

Comparison of Linear and Non-Linear Data Structures

Linear Data Structure	**Non-Linear Data Structure**
Data elements are arranged in a linear order where each element is attached to its previous and next adjacent.	Data elements are arranged in hierarchical or networked manner.
Single level is involved.	Multiple levels are involved.
Implementation is easy in comparison to non-linear data structure.	Implementation is complex in comparison to linear data structure.
Data elements can be traversed in one way only.	Data elements can be traversed in multiple ways (e.g., Depth-First, Breadth-First, In-order, Preorder, Postorder).
Examples : array, stack, queue, linked list, and their variants.	**Examples** : trees, graphs, skip list, hash map, and several variants.

Complexity of Common Data Structure Operations

	Data Structure	Time Complexity								Space Complexity
		Average				Worst				Worst
		Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Linear Data Structures	Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
	Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
	Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
	Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
	Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Non-Linear Data Structures	Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
	Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
	Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
	Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
	B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
	Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
	Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
	AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
	KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

Module Summary

- Introduced Non-linear Data Structures - graph, tree, hash table
- Studied Binary Search Tree as an adaptation of binary search
- Compared Linear and Non-Linear Data Structures

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.